Plano de Aula: CRUD React e Node.js

Objetivo: Construir um sistema de gerenciamento de produtos com arquitetura Cliente-Servidor. O backend (Node.js + Express) fornecerá uma API REST, e o frontend (React via Create React App) consumirá essas rotas. Os dados ficarão na memória do servidor (array) e serão perdidos ao reiniciar.

Divisão do Tempo (70 Minutos)

00-15 min: Setup de Pastas e Dependências

Criação das pastas separadas para backend e frontend, inicialização dos projetos e instalação de dependências (Express, Cors, Create React App).

15-30 min: Backend Node.js

Criação do arquivo `server.js` contendo as 5 rotas obrigatórias (GET, POST, PUT, DELETE) e inicialização do servidor local na porta 3001.

30-55 min: Frontend React (Lógica e Layout)

Limpeza do CRA, criação do layout CSS e programação do `App.js` para conectar com o servidor utilizando requisições `fetch`.

55-70 min: Integração e Testes

Execução simultânea de front e back em terminais separados, testes de uso e resolução de erros comuns de CORS e portas.

Passo 1: Criando as Pastas (15 min)

Crie uma pasta principal chamada `projeto-crud`, abra-a no seu editor de código (ex: VS Code) e abra o terminal. Faremos a configuração do servidor e do cliente em pastas separadas.

```
# 1. Configurando o Backend
mkdir backend
cd backend
npm init -y
npm install express cors

# Volte para a pasta principal (projeto-crud)
cd ..

# 2. Configurando o Frontend (Create React App)
npx create-react-app frontend

# Aguarde a instalação concluir (pode levar alguns minutos).
```

Passo 2: Construindo o Backend (15 min)

Na pasta `backend`, crie um arquivo chamado `server.js`. Cole o código abaixo. Ele define um servidor rodando na porta 3001, usando um array (`produtos`) como banco de dados temporário.

```
// backend/server.js
const express = require('express');
const cors = require('cors');

const app = express();
app.use(cors()); // Permite o React acessar a API
app.use(express.json()); // Permite ler dados no formato JSON

let produtos = []; // Array na memória (dados somem ao reiniciar)
let idAtual = 1;

// 1. GET /produtos - Listar todos os produtos
app.get('/produtos', (req, res) => {
  res.json(produtos);
});

// 2. GET /produtos/:id - Buscar um produto específico
app.get('/produtos/:id', (req, res) => {
  const produto = produtos.find(p => p.id == req.params.id);
  if (produto) {
    res.json(produto);
  } else {
    res.status(404).json({ erro: 'Produto não encontrado' });
  }
});

// 3. POST /produtos - Cadastrar novo produto
app.post('/produtos', (req, res) => {
  const { nome, preco } = req.body;
  if (!nome || !preco) {
    return res.status(400).json({ erro: 'Nome e preço são obrigatórios' });
  }

  const novoProduto = { id: idAtual++, nome, preco: Number(preco) };
  produtos.push(novoProduto);
  res.status(201).json(novoProduto); // 201 = Criado com sucesso
});

// 4. PUT /produtos/:id - Atualizar produto existente
app.put('/produtos/:id', (req, res) => {
  const { nome, preco } = req.body;
  const index = produtos.findIndex(p => p.id == req.params.id);

  if (index === -1) {
    return res.status(404).json({ erro: 'Produto não encontrado' });
  }

  produtos[index] = { ...produtos[index], nome, preco: Number(preco) };
  res.json(produtos[index]);
});

// 5. DELETE /produtos/:id - Excluir produto
app.delete('/produtos/:id', (req, res) => {
  produtos = produtos.filter(p => p.id != req.params.id);
  res.status(204).send(); // 204 = Sem conteúdo (sucesso na exclusão)
});
```

Passo 3: Visual do Frontend (10 min)

Agora vamos trabalhar na pasta `frontend`. Abra `frontend/src/App.css`, apague todo o conteúdo original criado pelo Create React App e cole o código abaixo. Em seguida, apague o arquivo `logo.svg` e limpe o `index.css`.

```
/* frontend/src/App.css */
.container {
  max-width: 700px;
  margin: 2rem auto;
  padding: 1rem;
  font-family: Arial, sans-serif;
  color: #333;
}

h1 { text-align: center; color: #4c1d95; margin-bottom: 2rem; }

.form-container {
  background: #f3f4f6;
  padding: 1.5rem;
  border-radius: 8px;
  margin-bottom: 2rem;
  border: 1px solid #e5e7eb;
}

.form-group { margin-bottom: 1rem; display: flex; flex-direction: column; }
.form-group label { font-weight: bold; margin-bottom: 0.5rem; }
.form-group input { padding: 0.8rem; border: 1px solid #d1d5db; border-radius: 4px; }

.btn {
  padding: 0.8rem 1.5rem; border: none; border-radius: 4px;
  cursor: pointer; font-weight: bold; color: white;
}
.btn-salvar { background: #10b981; width: 100%; margin-top: 10px; font-size: 1.1rem; }
.btn-salvar:hover { background: #059669; }
.btn-editar { background: #f59e0b; margin-right: 0.5rem; }
.btn-excluir { background: #ef4444; }

.lista-produtos { list-style: none; padding: 0; }
.produto-item {
  display: flex; justify-content: space-between; align-items: center;
  padding: 1rem; border-bottom: 1px solid #e5e7eb;
}
.produto-info span { display: block; }
.produto-nome { font-weight: bold; font-size: 1.1rem; }
.produto-preco { color: #059669; }

.erro-msg {
  background: #fef2f2; color: #b91c1c; padding: 10px;
  border-radius: 4px; margin-bottom: 1rem; border: 1px solid #f87171;
}
```

Design Simples: Substituímos a tabela por uma lista flexível (`.produto-item`), que naturalmente se adapta melhor a telas de celulares sem precisarmos de media queries complexas.

Passo 4: Lógica do Frontend (15 min) - Parte 1

Abra o arquivo `frontend/src/App.js`, apague o conteúdo e cole a lógica abaixo. Aqui fazemos as requisições (`fetch`) para o backend que construímos.

```
// frontend/src/App.js - Parte 1
import { useState, useEffect } from 'react';
import './App.css';

function App() {
  const [produtos, setProdutos] = useState([]);
  const [formData, setFormData] = useState({ id: null, nome: '', preco: '' });
  const [erro, setErro] = useState('');

  // URL base do nosso backend Node.js
  const API_URL = 'http://localhost:3001/produtos';

  // 1. READ: Carregar produtos ao abrir a página
  useEffect(() => {
    carregarProdutos();
  }, []);

  const carregarProdutos = async () => {
    try {
      const response = await fetch(API_URL);
      const data = await response.json();
      setProdutos(data);
    } catch (error) {
      setErro('Erro ao conectar com o servidor backend.');
    }
  };

  // 2. CREATE e UPDATE: Enviar formulário
  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!formData.nome || !formData.preco) {
      return setErro('Preencha nome e preço!');
    }

    const metodo = formData.id ? 'PUT' : 'POST';
    const url = formData.id ? `${API_URL}/${formData.id}` : API_URL;

    try {
      await fetch(url, {
        method: metodo,
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ nome: formData.nome, preco: formData.preco })
      });

      setFormData({ id: null, nome: '', preco: '' }); // Limpa form
      setErro('');
      carregarProdutos(); // Atualiza a lista
    } catch (error) {
      setErro('Erro ao salvar produto.');
    }
  };

  // 3. DELETE: Excluir produto
  const handleDelete = async (id) => {
    if (window.confirm('Excluir este produto?')) {
      try {
```

Passo 4: Interface do Frontend (15 min) - Parte 2

Continue no arquivo `frontend/src/App.js` e cole o código de renderização logo abaixo das funções que acabamos de criar. Este JSX desenha os campos e a lista na tela.

```
// (Continuação de frontend/src/App.js)

return (
  <div className="container">
    <h1>Gerenciador de Produtos</h1>

    <div className="form-container">
      {erro && <div className="erro-msg">{erro}</div>}

      <form onSubmit={handleSubmit}>
        <div className="form-group">
          <label>Nome do Produto</label>
          <input
            type="text"
            value={formData.nome}
            onChange={(e) => setFormData({...formData, nome: e.target.value})}
            placeholder="Ex: Teclado"
          />
        </div>

        <div className="form-group">
          <label>Preço (R$)</label>
          <input
            type="number"
            step="0.01"
            value={formData.preco}
            onChange={(e) => setFormData({...formData, preco: e.target.value})}
            placeholder="Ex: 150.00"
          />
        </div>

        <button type="submit" className="btn btn-salvar">
          {formData.id ? 'Atualizar Produto' : 'Cadastrar Produto'}
        </button>
      </form>
    </div>

    <h2>Lista de Produtos</h2>
    {produtos.length === 0 ? (
      <p>Nenhum produto cadastrado no servidor.</p>
    ) : (
      <ul className="lista-produtos">
        {produtos.map((produto) => (
          <li key={produto.id} className="produto-item">
            <div className="produto-info">
              <span className="produto-nome">{produto.nome}</span>
              <span className="produto-preco">
                R$ {Number(produto.preco).toFixed(2)}
              </span>
            </div>
            <div className="produto-acoes">
              <button onClick={() => handleEdit(produto)} className="btn btn-editar">
                Editar
              </button>
              <button onClick={() => handleDelete(produto.id)} className="btn btn-excluir">
                Excluir
```

Passo 5: Integração e Testes (10 min)

Para o sistema funcionar, você precisa de **dois terminais** abertos simultaneamente. Um rodando o Node.js e outro rodando o React.

```
# Terminal 1 (Backend):
cd backend
node server.js

# Terminal 2 (Frontend):
cd frontend
npm start
```

O navegador abrirá automaticamente em `http://localhost:3000`. Siga o checklist para garantir que a comunicação está perfeita:

Checklist de Testes:

- [ ] **Create (POST):** Preencha nome e preço, clique em Cadastrar. O produto deve aparecer na lista instantaneamente.
- [ ] **Read (GET):** Atualize a página do navegador (F5). O produto não deve sumir (foi recarregado da memória do servidor).
- [ ] **Update (PUT):** Clique em Editar, mude o preço e Atualize. Verifique na lista.
- [ ] **Delete (DELETE):** Clique em Excluir e confirme.
- [ ] **Perda de Dados:** No Terminal 1 (Backend), aperte `Ctrl+C` para parar o servidor e rode `node server.js` novamente. Atualize o navegador: a lista ficará vazia (comportamento esperado de arrays em memória).

Resolução de Erros Comuns

Problema	Causa e Solução
Erro "Failed to fetch" ou "Erro ao conectar"	O servidor backend não está rodando. Certifique-se de executar `node server.js` na pasta do backend.
Erro de CORS no console	Você esqueceu de usar o `app.use(cors())` no `server.js` ou de instalar o pacote cors.
Novos produtos salvam vazios	Faltou a linha `app.use(express.json())` no `server.js`, então o Express não conseguiu ler o `req.body`.